

```

"""
Triangle drawer - AP CSP Create Task - Siwei Li
Draws triangles with turtle based on what the user enters (how many,
position, size, angle).
I use the Law of Cosines to figure out the third side so the triangle has
the third leg.
"""

import turtle
import math

def drawTriangle(base, side, angle):
    """
    Draws one triangle. Takes base length, one side length, and the angle
    between them. I use a procedure so I can call it over and over with
    different numbers instead of copying the same drawing code everywhere.
    """
    start_x = turtle.xcor()
    start_y = turtle.ycor()

    # Law of Cosines to get the third side:  $c^2 = a^2 + b^2 - 2ab\cos(\text{angle})$ 
    #  $\cos()$  wants radians not degrees so I convert
    angle_radians = math.radians(angle)
    third_side_squared = base**2 + side**2 - 2 * base * side *
    math.cos(angle_radians)

    # if this is negative the sides don't make a real triangle so skip it
    if third_side_squared <= 0:
        print(" [Skipping] Those dimensions do not form a valid
triangle.")
        return

    third_side = math.sqrt(third_side_squared)

    # draw all 3 sides - use a loop (iteration)
    for side_num in range(3):
        if side_num == 0:
            # first side is just the base

```

```

        turtle.forward(base)
    elif side_num == 1:
        # turn then draw the second side (the 180-angle thing gives
the right interior angle)
        turtle.left(180 - angle)
        turtle.forward(side)
    else:
        # third side - point back to start and draw the line we
calculated
        turtle.setheading(turtle.towards(start_x, start_y))
        turtle.forward(third_side)

    # face right again so the next triangle isn't tilted
    turtle.setheading(0)

def main():
    # set up the window and turtle
    screen = turtle.Screen()
    screen.title("AP CSP Create Task - Triangle Drawer")
    screen.setup(width=800, height=600)
    screen.bgcolor("white")
    turtle.speed(3)
    turtle.pensize(2)
    turtle.pencolor("darkblue")
    turtle.penup()
    turtle.goto(-200, 0)
    turtle.pendown()

    # list to hold all the triangle info - that way I can have however
many triangles without making triangle1, triangle2, triangle3 etc.
(abstraction)
    list_of_triangles = []

    try:
        num_triangles = int(input("Enter the number of triangles to draw
(e.g., 2): "))
    except ValueError:
        num_triangles = 2

```

```

        print("Using default: 2 triangles.")

    # ask for each triangle's position and dimensions
    for i in range(num_triangles):
        print(f"\nTriangle {i + 1}")
        try:
            x_pos = float(input("Enter x coordinate to draw at (e.g.,
-200): "))
            y_pos = float(input("Enter y coordinate to draw at (e.g., 0):
"))
            base_len = float(input("Enter base length (e.g., 80): "))
            side_len = float(input("Enter side length (e.g., 80): "))
            angle_deg = float(input("Enter angle between base and side in
degrees (e.g., 60): "))
        except ValueError:
            # if they type something that's not a number use defaults
            x_pos, y_pos = -200 + i * 120, 0
            base_len, side_len, angle_deg = 60, 60, 60
            print("Using defaults for position and dimensions.")
        list_of_triangles.append((x_pos, y_pos, base_len, side_len,
angle_deg))

    # go through the list and draw each one at its x,y
    for triangle_data in list_of_triangles:
        x_val, y_val, base_val, side_val, angle_val = triangle_data
        turtle.penup()
        turtle.goto(x_val, y_val)
        turtle.pendown()
        drawTriangle(base_val, side_val, angle_val)

    turtle.penup()
    turtle.goto(0, -150)
    turtle.write("Close the window to exit.", align="center",
font=("Arial", 12, "normal"))
    screen.exitonclick()

if __name__ == "__main__":
    main()

```

